

IFN647 Advanced Information Retrieval — Exam Review Notes

Exam format: 13 questions, 45 marks total. Q1–Q5 are short answer (10 marks), Q6–Q13 are problem solving (35 marks). Covers Weeks 2–12. A few questions require short Python code.

MODULE 1 — Text Pre-processing, IR, IE & Text Indexing (Weeks 2–4)

1.1 Text Statistics & Pre-processing (Week 2)

Term frequency: Open a file, tokenize into words, count occurrences using a dictionary. Filter terms by length > 2. Sort in descending order of frequency.

Stemming reduces words to a common stem via inflection (plurals, tenses) or derivation (suffixation).

- **Algorithmic stemmer** (e.g., Porter): uses a small program based on suffix rules to decide if two words are related.
- **Dictionary-based stemmer:** relies on pre-created dictionaries of related terms — has **no logic of its own**.
- *Exam trap:* Saying a dictionary-based stemmer "uses a small program based on suffix knowledge" is **FALSE** — that describes an algorithmic stemmer.

N-grams: Overlapping sequences of words. Bigrams = 2-word windows, trigrams = 3-word windows. Move the window one word at a time.

HTML parsing: Use `HTMLParser` to extract hyperlinks. Ethical issues include potential DoS attacks from excessive requests and possible copyright/terms-of-service violations.

1.2 Retrieval Models (Week 3)

Boolean Retrieval: Exact-match model — documents retrieved if they exactly match the query specification. Limitation: searching by numbers (focus on retrieved set size).

Vector Space Model:

- Documents and queries are t -dimensional vectors (t = number of index terms).
- There is **no explicit definition of relevance** — only an implicit assumption that relevance relates to similarity of query and document vectors.
- *Exam trap*: "There is an explicit definition of relevance in the vector space model" is **FALSE**.

Document Features:

- **Topical features**: relate to the topic/content of the document.
- **Quality features**: meta-information (e.g., days since last update).
- *Exam trap*: "Topical features are the only features" is **FALSE** — quality features also exist.

Bayes Classifier (Binary Independence Model): Given query Q and collection C with relevant documents R :

- Let $p_i = P(d_i | R)$ = fraction of relevant documents containing term d_i
- $P(D|R)$ = product of p_i for all query terms present in D

Example: $Q = \{US, ECONOM, ESPIONAG\}$, relevant docs = $\{D2, D3, D4\}$

- $P(US|R) = 3/3 = 1.0$, $P(ECONOM|R) = 3/3 = 1.0$, $P(ESPIONAG|R) = 2/3 = 0.667$
- $P(D|R) = 1.0 \times 1.0 \times 0.667 = \mathbf{0.667}$

Term weighting formula: $w1(t) = \log[(r(t)/R) / (n(t)/N)]$

- $n(t)$ = number of documents containing term t
- $r(t)$ = number of **relevant** documents containing term t
- N = total documents, R = total relevant documents

Abstract Model of Ranking: $R(Q, D)$ compares document with query to compute a score. To convert to pairwise $f(A, B, Q)$: $f(A, B, Q) = 1$ if $R(Q, A) > R(Q, B)$, else -1 .

Word2Vec: Uses NLTK and Gensim. Tokenize text $\rightarrow$ sentences $\rightarrow$ words. Train CBOW/Skip-gram model. Use `wv.similarity()` for semantic similarity.

1.3 Inverted Indexing & Text Indexing (Week 4)

Inverted Index: Associates each index term with a list of (document ID, term count) pairs. The index is "inverted" because documents are associated with words (instead of words being part of documents).

- *Exam trap*: A binary inverted index (term present/absent) does **NOT** contain enough information to find exact phrases like "tropical fish" — you need **position information** for that.

Postings: Each list entry is a posting; the part referring to a specific document is called a pointer.

Extents: Contiguous regions of a document, represented using word positions.

Term-at-a-Time Algorithm: Process one inverted list at a time, accumulating partial scores for each document.

Example steps:

1. Process L1 (freshwater): partial scores {1:1, 4:1}
2. Process L2 (aquarium): add → {1:1, 3:1, 4:1}
3. Process L3 (fish): add → {1:3, 2:3, 3:3, 4:3}
4. Process L4 (tropical): add → final scores {1:5, 2:5, 3:4, 4:3}

Markov Chains: Transition matrix is a list of lists of probabilities. Each row sums to 1.

Example: Sleep→Run probability = 0.6 (60%). Matrix: [[0.2,0.6,0.2], [0.1,0.6,0.3], [0.2,0.7,0.1]]

LLM-based Information Extraction: Design prompts to extract structured entities from XML documents. Return results as a valid Python dictionary.

MODULE 2 — Language Models, Information Needs & Evaluation (Weeks 5–6)

2.1 Language Models (Week 5)

Jelinek-Mercer Smoothing:

$$P(Q|D) = \prod P(q_i|D) \text{ for all query terms } q_i$$

$$P(q_i|D) = (1-\lambda) \times (fq_{i,D} / |D|) + \lambda \times (cq_i / |C|)$$

where $fq_{i,D}$ = count of q_i in D , $|D|$ = document length, cq_i = count of q_i in collection, $|C|$ = total words in collection.

Example: $\lambda = 0.4$, $D2 = \{US, US, ECONOM, SPY\}$, $Q = \{US, ECONOM, ESPIONAG\}$

- $P(US|D2) = 0.6 \times (2/4) + 0.4 \times (4/23) = 0.368$
- $P(ECONOM|D2) = 0.6 \times (1/4) + 0.4 \times (3/23) = 0.202$
- $P(ESPIONAG|D2) = 0.6 \times (0/4) + 0.4 \times (3/23) = 0.052$
- **$P(Q|D2) = 0.368 \times 0.202 \times 0.052 = 0.003865$**

2.2 Evaluation Metrics (Week 5)

Query Logs: A typical query log **does** contain user identifier/session identifier, query terms, URLs, ranks, clicks, timestamps. Privacy is an issue when logs are shared/distributed.

- Exam trap: "A typical query log does not contain user identifier" is **FALSE**.

Pooling: Selects top-k results from different retrieval algorithms, merges into a pool, removes duplicates, presents in random order for relevance judging. The list is incomplete — problematic for new algorithms not in the original pool.

Sets:

- A = relevant documents, A' = non-relevant
- B = retrieved documents, B' = not retrieved

Recall = $|A \cap B| / |A|$

Precision = $|A \cap B| / |B|$

False Positive (Fallout) = $|A' \cap B| / |A'|$

False Negative = $|A \cap B'| / |A|$

F1 Measure = $2 \times \text{Recall} \times \text{Precision} / (\text{Recall} + \text{Precision})$

Example: $A = \{110, 111, 113, 114, 116, 117, 119, 123\}$, $B = \{111, 112, 113, 110, 114, 119\}$

- $A \cap B = \{110, 111, 113, 114, 119\} \rightarrow \text{Recall} = 5/8 = 62.5\%$, $\text{Precision} = 5/6 = 83.33\%$
- $F1 = 2 \times (5/8) \times (5/6) / (5/8 + 5/6) = \mathbf{71.43\%}$

2.3 Spelling Correction & Advanced Metrics (Week 6)

Edit Distance (Damerau-Levenshtein): Counts the **minimum** number of insertions, deletions, substitutions, or transpositions.

- Exam trap: "maximum number" is **FALSE** — it's the **minimum**.

Noisy Channel Model: $P(w|e) \propto P(e|w) \times P(w)$

Using language model: $P(w) = \lambda P(w) + (1-\lambda)P(w|w_prev)$

Example: "fish tink" → correcting "tink": $P(\text{tank}|\text{fish}) > P(\text{think}|\text{fish})$ in most corpora, so "tank" is more likely correction despite equal edit distances and priors.

Precision at Rank k (P@k): Number of relevant docs in top k positions / k.

Average Precision (AP): Average of precision values computed at each position where a relevant document is retrieved.

Example: Ranked output: 111 112 113 110 114 119 115 122 118 116 123 121 120 117 124

$$- \text{AP} = (1 + 2/3 + 3/4 + 4/5 + 5/6 + 6/10 + 7/11 + 8/14) / 8 = \mathbf{73.21\%}$$

MAP (Mean Average Precision): Average of AP values across multiple queries.

Interpolated Precision at Standard Recall Levels: $P(R) = \max\{P' : R' \geq R \wedge (R', P') \in S\}$

Python: `[max([p for (r,p) in S if r >= i/10]) for i in range(11)]`

MODULE 3 — Learning to Rank, User & Relevance Models (Weeks 7–8)

3.1 Mutual Information Measure (Week 7)

$$\text{MIM}(a, b) = \log[P(a,b) / (P(a) \times P(b))]$$

Using document frequencies:

- $P(a) = \text{df}(D, a)/N$, $P(b) = \text{df}(D, b)/N$, $P(a,b) = \text{df2}(D, a, b)/N$
- $\text{MIM} = \log[N \times \text{df2}(D, a, b) / (\text{df}(D, a) \times \text{df}(D, b))]$

df2 function: Count documents containing both terms a and b:

def df2(docs, a, b):

nab = len([d for d in docs if a in docs[d] and b in docs[d]])

return nab

3.2 Term Frequency Measure (Week 7)

$tf(t) = \text{sum of } t\text{'s frequency in relevant documents} / \text{sum of all term frequencies in relevant documents}$

Example: ESPIONAG in $D^+ = \{D2, D3, D4\}$: $tf = (0+1+1)/(4+4+4) = 2/12 = \mathbf{0.167}$

3.3 BM25-based IF Model (Week 7)

$w5(t) = \log[(r(t)+0.5)(N-n(t)-R+r(t)+0.5) / ((n(t)-r(t)+0.5)(R-r(t)+0.5))]$

where:

- $N = \text{len}(D)$ — total documents in training set
- $R = \text{len}(\text{Rel})$ — number of relevant documents
- $n(t) = \text{len}([d \text{ for } d \text{ in } D \text{ if } t \text{ in } d])$ — documents containing term t
- $r(t) = \text{len}([d \text{ for } d \text{ in } \text{Rel} \text{ if } t \text{ in } d])$ — relevant documents containing term t

3.4 LLMs and Information Filtering (Week 7)

- LLMs generate human-readable explanations for recommendations (transparency).
- LLM-based filtering doesn't require labelled training data — prompts guide behavior.
- **RAG** retrieves external knowledge to produce more accurate outputs.
- **CoT (Chain-of-Thought)** prompting: step-by-step reasoning before final answer.
- *Exam trap:* "LLM effectiveness in real-world news recommendation has been **fully established**" is **FALSE** — it has **yet to be** fully established.

3.5 Query Likelihood & Pseudo Relevance Feedback (Week 8)

Query Likelihood Model: Documents ranked by $P(Q|D)$ — probability query text generated by document language model.

KL Divergence: Measures difference between two probability distributions. It is **NOT** a smoothing technique.

- *Exam trap:* "KL divergence is a technique for overcoming data sparsity" is **FALSE** — **smoothing** does that. KL divergence measures distributional difference.

Smoothing: Technique for avoiding estimation problems due to data sparsity.

Pseudo-Relevance Feedback Pipeline:

1. Rank documents using initial query Q with an IR model
2. Select top-k1 documents as pseudo-relevant (Rel1)
3. Extract important topical features (high tf×idf terms)
4. Use KL-divergence to re-rank, select top-k2 as Rel2
5. Build training set: D+ = Rel2, D- = C - Rel2
6. Apply probabilistic IF model with term weighting formulas

Query Likelihood for Tweets: $s(d,Q) = \sum c(w,Q) \times \log(1 + c(w,d)/(\mu \times P(w|C))) + |Q| \times \log(\mu/(\mu+|d|))$

- $c(w,d)$ = count of word w in tweet d
- $c(w,Q)$ = count of word w in query Q
- $P(w|C)$ = probability of word w in collection C (for normalization)

MODULE 4 — Text Classification & Clustering (Weeks 9–11)

4.1 Naïve Bayes Classification (Week 9)

Prior probability: $P(c) = N_c / |D|$

Conditional probability (Laplacian smoothing): $P(w|c) = (tf_{\{w,c\}} + 1) / (|c| + |V|)$

where $tf_{\{w,c\}}$ = times term w occurs in class c, $|c|$ = total term occurrences in class c, $|V|$ = vocabulary size.

Classification: $P(c|d) = \log P(c) + \sum \log P(w_i|c)$ for all words w_i in d. Assign class with highest score.

Example: Spam classification with $V = \{\text{cheep, buy, banking, dinner, the}\}$

- $P(\text{spam}) = 3/10 = 0.3$, $P(\text{not_spam}) = 7/10 = 0.7$
- $P(\text{cheep}|\text{spam}) = (10+1)/(20+5) = 11/25 = 0.44$
- $P(\text{cheep}|\text{not_spam}) = (1+1)/(15+5) = 2/20 = 0.10$

4.2 Rocchio Classification (Week 9)

Uses centroids to define class boundaries. Centroid = vector average of all members in a class.

For each class, sum all document vectors and divide by the number of documents in that class.

Example: Centroid of non-relevant class = average of feature weights across all non-relevant documents.

4.3 Sentiment Analysis (Week 9)

Key concepts:

- **Orientation:** The opinion about an entity/aspect by the opinion holder at a specific time.
- **Aspects:** Features, components, or functions of the entity (nouns/noun phrases).
- **Polarity classification:** Groups opinion into positive, negative, or neutral.
- **Subjectivity classification:** Separates subjective from objective information (binary task).
- **Emotion classification:** Classifies text by predefined basic emotions.
- *Exam trap:* "Emotion classification separates subjective from objective information" is **FALSE** — that's **subjectivity classification**.

Aspect-Based Sentiment Analysis (ABSA):

- Method A: NLTK POS tagging — extract tokens with tags starting with "NN" (nouns).
- Method B: LLM prompt — more accurate but less transparent.
- Trade-off: A is interpretable but weaker; B is more effective but lacks explainability.

4.4 Social Media & Recommender Systems (Week 10)

Social Media Analytics: Extracting insights from semi-structured/unstructured social media data.

Social Search: Search involving communities (tagging, Q&A). User interaction data (tags, collaborative search) provides unique data sources.

Adaptive Filtering: Long-term information needs expressed as profiles. Key challenge: high recall (don't miss relevant docs) AND high precision (don't alert to irrelevant docs).

Static Filtering (Language Model): Profile word probability distribution with smoothing, then use negative KL divergence between profile and document models for relevance scoring.

Collaborative Filtering:

- Users represented by rating vectors over items (0 = unrated).
- `my_correlation(Rvs)` → returns users, items, correlation matrix.
- `my_cluster(Rvs)` → returns user clusters.
- `my_prediction(Rvs, C, U, I)` → predicts ratings for unseen items.

4.5 Clustering & Distance Functions (Week 11)

Binary Contingency Table: For two documents, count:

- q = both have 1, r = $d1=1$ & $d2=0$, s = $d1=0$ & $d2=1$, t = both have 0

Jaccard Distance: $\text{disJac}(d1, d2) = (r + s) / (q + r + s)$

Simple Matching Coefficient Distance: $\text{disS}(d1, d2) = (r + s) / (q + r + s + t)$

Example: $d1=(1,1,0,0,0,0,0)$, $d2=(0,0,1,1,0,1,0) \rightarrow q=0, r=2, s=3, t=2$

- Jaccard = $5/5 = 1.0$, SMC = $5/7 = 0.714$

Cluster Precision:

- $\text{MaxClass}(C_i)$ = largest set of documents in cluster C_i that share the same true class label.
- Cluster Precision = $\sum |\text{MaxClass}(C_i)| / |D|$

Example: $C1=\{d2,d5,d10\}$, $C2=\{d1,d3,d4,d6,d7,d8,d9\}$

- $\text{MaxClass}(C1) = \{d2,d5\}$ (spam), $\text{MaxClass}(C2) = \{d1,d3,d6,d7,d8,d9\}$ (not_spam)
- Precision = $(2+6)/10 = 0.8$

MODULE 5 — Web Search, Feature Dependencies & Multimodal Learning (Week 12)

5.1 PageRank

$\text{PR}(u) = \lambda/N + (1-\lambda) \times \sum \text{PR}(v)/L_v$ for all $v \in B_u$

where B_u = pages pointing to u , L_v = number of outgoing links from v , λ = damping factor (typically 0.15), N = total pages.

Example: Graph: $A \rightarrow B$, $A \rightarrow D$, $C \rightarrow A$, $C \rightarrow B$, $D \rightarrow B$. Initial PR = 0.25 each, $\lambda=0.15$.

First iteration:

- $B_A = \{C\}, L_C = 2 \rightarrow \mathbf{PR(A) = 0.15/4 + 0.85 \times (0.25/2) = 0.14375}$
- $B_B = \{A, C, D\}, L_A=2, L_C=2, L_D=1 \rightarrow \mathbf{PR(B) = 0.0375 + 0.85 \times (0.25/2 + 0.25/2 + 0.25) = 0.4625}$
- $B_C = \{\} \rightarrow \mathbf{PR(C) = 0.15/4 = 0.0375}$
- $B_D = \{A\}, L_A=2 \rightarrow \mathbf{PR(D) = 0.0375 + 0.85 \times (0.25/2) = 0.14375}$

5.2 BERT & Attention Mechanism

Steps to understand attention_mask in BERT:

1. Tokenize input text
2. Generate initial embeddings (encoder inputs)
3. Pass to BERT encoder with attention_mask $\rightarrow$ obtain encoder outputs
4. Compare token representations before and after encoding (attention changes them)

5.3 Contrastive Learning with Pseudo-Relevance Feedback

Pipeline:

1. Use baseline IR model (e.g., BM25) to retrieve top-k documents for query Q
2. Treat top-k as pseudo-positives, low-ranked/random as negatives
3. Form training pairs: (Q, positive_doc) and (Q, negative_doc)
4. Train encoder using contrastive loss (e.g., InfoNCE/SimCLR)
5. Re-encode and re-rank documents using trained encoder

Contrastive Loss (SimCLR-style):

1. Normalize z_1, z_2 to unit vectors
 2. Merge into $z = [z_1; z_2]$
 3. Compute similarity matrix $\text{sim}(z_i, z_j) / \text{temperature}$
 4. Build labels for positive pairs
 5. Remove self-similarity (set diagonal to $-\infty$)
 6. Extract positive pair logits
 7. Compute loss: $L_i = -\text{sim}(q, d+) + \log(\sum \exp(\text{sim}(q, d_j)/\tau))$
 8. Return mean loss
-

Quick Reference — Common "FALSE" Answers

Week	FALSE Statement	Correct Answer
2	Dictionary-based stemmer uses a program based on suffix knowledge	It uses pre-created dictionaries; algorithmic stemmers use programs
3	Explicit definition of relevance in vector space model	Only implicit assumption (similarity)
3	Topical features are the only features	Quality features also exist
4	Binary inverted index can find exact phrases	Needs position information
5	Query log doesn't contain user identifier	It does; privacy is an issue when shared
6	Damerau-Levenshtein counts maximum operations	It counts the minimum
7	LLM effectiveness in news recommendation fully established	Yet to be fully established
8	KL divergence overcomes data sparsity	Smoothing does that; KL measures distributional difference
9	Emotion classification separates subjective from objective	That's subjectivity classification

Key Python Patterns to Know

Term frequency in relevant docs

```
tf_t = sum(d.count(t) for d in Rel) / sum(len(d) for d in Rel)
```

Document frequency

$n_t = \text{len}([d \text{ for } d \text{ in } D \text{ if } t \text{ in } d])$

Relevant document frequency

$r_t = \text{len}([d \text{ for } d \text{ in } Rel \text{ if } t \text{ in } d])$

Naïve Bayes conditional probability (Laplacian smoothing)

$P_{w_c} = (tf_{wc} + 1) / (total_terms_in_c + vocab_size)$

List comprehension for interpolated precision

$[\max([p \text{ for } (r,p) \text{ in } S \text{ if } r \geq i/10]) \text{ for } i \text{ in range}(11)]$

Cosine similarity

$\text{cos_sim} = \text{dot}(a, b) / (\text{norm}(a) * \text{norm}(b))$

Good luck with the exam!